13 Coding Standards Part 2: new_script

In Part 1, we created a coding standard that will assist us when writing serious, production-quality scripts. The only problem is the standard is rather complicated, and writing a script that conforms to it can get a bit tedious. Any time we want to write a "good" script, we have to do a lot of rote, mechanical work to include such things as error handlers, traps, help message functions, etc.

To overcome this, many programmers rely on script templates that contain much of this routine coding. In this adventure, we're going to look at a program called `new_script` from LinuxCommand.org that creates templates for bash scripts. Unlike static templates, `new_script` custom generates templates that include usage and help messages, as well as a parser for the script's desired command line options and arguments. Using `new_script` saves a lot of time and effort and helps us make even the most casual script a well-crafted and robust program.

Installing new_script

To install `new_script`, we download it from LinuxCommand.org, move it to a directory in our PATH, and set it to be executable.

```
me@linuxbox:~$ curl -O http://linuxcommand.org/new_script.bash
me@linuxbox:~$ mv new_script.bash ~/bin/new_script
me@linuxbox:~$ chmod +x ~/bin/new_script
```

After installing it, we can test it this way:

```
me@linuxbox:~$ new_script --help
```

If the installation was successful, we will see the help message:

```
new_script 3.5.3
Bash shell script template generator.

Usage: new_script [-h|--help ]
       new_script [-q|--quiet] [-s|--root] [script]

  Options:

  -h, --help    Display this help message and exit.
  -q, --quiet   Quiet mode. No prompting. Outputs default script.
  -s, --root    Output script requires root privileges to run.
```

Options and Arguments

Normally, `new_script` is run without options. It will prompt the user for a variety of information that it will use to construct the script template. If an output script file name is not specified, the user will be prompted for one. For some special use cases, the following options are supported:

- **-h, –help** The help option displays the help message we saw above. The help option is mutually exclusive with the other `new_script` options and after the help message is displayed, `new_script` exits.
- **-q, –quiet** The quiet option causes `new_script` to become non-interactive and to output a base template without customization. In this mode, `new_script` will output its script template to standard output if no output script file is specified.
- **-s, –root** The superuser option adds a routine to the template that requires the script to be run by the superuser. If a non-privileged user attempts to run the resulting script, it will display an error message and terminate.

Creating Our First Template

Let's make a template to demonstrate how `new_script` works and what it can do. First, we'll launch `new_script` and give it the name of a script we want to create.

```
me@linuxbox:~$ new_script new_script-demo

----------------------------------------------------------------------
** Welcome to new_script version 3.5.3 **
----------------------------------------------------------------------

File 'new_script-demo' exists. Overwrite [y/n] > y
```

We'll be greeted with a welcome message. If the script already exists, we are prompted to overwrite. If we had not specified a script file name, we would be prompted for one.

```
----------------------------------------------------------------------
** Comment Block **

The purpose is a one line description of what the script does.
----------------------------------------------------------------------
The purpose of the script is to: > demonstrate the new_script template

----------------------------------------------------------------------
The script may be licensed in one of two ways:
1. All rights reserved (default) or
2. GNU GPL version 3 (preferred).
----------------------------------------------------------------------

Include GPL license header [y/n]? > y
```

The first information `new_script` asks for are the purpose of the script and how it is licensed. Later, when we examine the finished template below, we'll see that `new_script` figures out the author's name and email address, as well as the copyright date.

```
----------------------------------------------------------------------
** Privileges **

The template may optionally include code that will prevent it from
running if the user does not have superuser (root) privileges.
----------------------------------------------------------------------
```

```
Does this script require superuser privileges [y/n]? > n
```

If we need to make this script usable only by the superuser, we set that next.

```
------------------------------------------------------------------------
** Command Line Options **

The generated template supports both short name (1 character), and long
name (1 word) options. All options must have a short name. Long names
are optional. The options 'h' and 'help' are provided automatically.

Further, each option may have a single argument. Argument names must
be valid variable names.

Descriptions for options and option arguments should be short (less
than 1 line) and will appear in the template's comment block and
help_message.
------------------------------------------------------------------------

Does this script support command-line options [y/n]? > y
```

Now we get to the fun part; defining the command line options. If we answer no to this question, `new_script` will write the template and exit.

As we respond to the next set of prompts, remember that we are building a help message (and a parser) that will resemble the `new_script` help message, so use that as a guide for context. Keep responses clear and concise.

```
Option 1:
  Enter short option name [a-z] (Enter to end) -> a
  Description of option ------------------------> the first option named 'a'
  Enter long option name (optional) ------------> option_a
  Enter option argument (if any) --------------->

Option 2:
  Enter short option name [a-z] (Enter to end) -> b
  Description of option ------------------------> the second option named 'b'
  Enter long option name (optional) ------------> option_b
  Enter option argument (if any) ---------------> b_argument
  Description of argument (if any)--------------> argument for option 'b'

Option 3:
  Enter short option name [a-z] (Enter to end) ->
```

By entering nothing at the short option prompt, `new_script` ends the input of the command options and writes the template. We're done!

A note about short option names: `new_script` will accept any value, not just lowercase letters. This includes uppercase letters, numerals, etc. Use good judgment.

A note about long option names and option arguments: long option names and option arguments must be valid bash variable names. If they are not, `new_script` will attempt correct them, If there are embedded spaces, they will be replaced with underscores. Anything else will cause `no_script` to replace the name with a calculated default value based on the short option name.

Looking at the Template

Here we see a numbered listing of the finished template.

```
 1  #!/usr/bin/env bash
 2  # ---------------------------------------------------------------------
 3  # new_script-demo - Demonstrate the new_script template

 4  # Copyright 2021, Linux User <me@linuxbox.example.com>
 5
 6  # This program is free software: you can redistribute it and/or modify
 7  # it under the terms of the GNU General Public License as published by
 8  # the Free Software Foundation, either version 3 of the License, or
 9  # (at your option) any later version.

10  # This program is distributed in the hope that it will be useful,
11  # but WITHOUT ANY WARRANTY; without even the implied warranty of
12  # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
13  # GNU General Public License at <http://www.gnu.org/licenses/> for
14  # more details.

15  # Usage: new_script-demo [-h|--help]
16  #        new_script-demo [-a|--option_a] [-b|--option_b b_argument]

17  # Revision history:
18  # 2021-05-05 Created by new_script ver. 3.5.3
19  # ---------------------------------------------------------------------
```

The comment block is complete with license, usage, and revision history. Notice how the first letter of the purpose has been capitalized and the author's name and email address have been calculated. `new_script` gets the author's name from the `/etc/password` file. If the `REPLYTO` environment variable is set, it supplies the email address (this was common with old-timey email programs); otherwise the email address will be expanded from `$USER@$(hostname)`. To define the `REPLYTO` variable, we just add it to our `~/.bashrc` file. For example:

```
export REPLYTO=me@linuxbox.example.com
```

Our script template continues with the constants and functions:

```
20  PROGNAME=${0##*/}
21  VERSION="0.1"
22  LIBS=     # Insert pathnames of required external shell libraries here
```

The global constants appear next, with the program name (derived from `$0`) and the version number. The `LIBS` constant should be set to contain a space-delimited list (in double quotes of course) of any files to be sourced. Note: the way the template implements this feature requires that library pathnames do not contain spaces. Besides the template not working, including embedded spaces in a library name would be in extremely poor taste.

```
23  clean_up() { # Perform pre-exit housekeeping
24    return
25  }
```

```
error_exit() {
  local error_message="$1"
  printf "%s: %s\n" "$PROGNAME" "${error_message:-"Unknown Error"}" >&2
  clean_up
  exit 1
}
graceful_exit() {
  clean_up
  exit
}
signal_exit() { # Handle trapped signals
  local signal="$1"
  case "$signal" in
    INT)
      error_exit "Program interrupted by user" ;;
    TERM)
      error_exit "Program terminated" ;;
    *)
      error_exit "Terminating on unknown signal" ;;
  esac
}
```

The first group of functions handles program termination. The `clean_up` function should include the code for any housekeeping tasks needed before the script exits. This function is called by all the other exit functions to ensure an orderly termination.

```
load_libraries() { # Load external shell libraries
  local i
  for i in $LIBS; do
    if [[ -r "$i" ]]; then
      source "$i" || error_exit "Library '$i' contains errors."
    else
      error_exit "Required library '$i' not found."
    fi
  done
}
```

The `load_libraries` function loops through the contents of the `LIBS` constant and sources each file. If any file is missing or contains errors, this function will terminate the script with an error.

```
usage() {
  printf "%s\n" "Usage: ${PROGNAME} [-h|--help]"
  printf "%s\n" \
    "       ${PROGNAME} [-a|--option_a] [-b|--option_b b_argument]"
}
help_message() {
  cat <<- _EOF_
$PROGNAME ver. $VERSION
Demonstrate the new_script template
```

```
$(usage)

  Options:
  -h, --help                  Display this help message and exit.
  -a, --option_a              The first option named 'a'
  -b, --option_b b_argument   The second option named 'b'
    Where 'b_argument' is the argument for option 'b'.

_EOF_
  return
}
```

The `usage` and `help_message` functions are based on the information we supplied. Notice how the help message is neatly formatted and the option descriptions are capitalized as needed.

```
# Trap signals
trap "signal_exit TERM" TERM HUP
trap "signal_exit INT"  INT

load_libraries
```

The last tasks involved with set up are the signal traps and calling the function to source the external libraries, if there are any.

Next comes the parser, again based on our command options.

```
# Parse command-line
while [[ -n "$1" ]]; do
  case "$1" in
    -h | --help)
      help_message
      graceful_exit
      ;;
    -a | --option_a)
      echo "the first option named 'a'"
      ;;
    -b | --option_b)
      echo "the second option named 'b'"
      shift; b_argument="$1"
      echo "b_argument == $b_argument"
      ;;
    --* | -*)
      usage >&2
      error_exit "Unknown option $1"
      ;;
    *)
      printf "Processing argument %s...\n" "$1"
      ;;
  esac
  shift
done
```

The parser detects each of our specified options and provides a simple stub for our actual code. One feature of the parser is that positional parameters that appear after the options are assumed to be arguments to the script so this template is ready to handle them even if the script has no options.

```
103  # Main logic

104  graceful_exit
```

We come to the end of the template where the main logic is located. Since this script doesn't do anything yet, we simply call the `graceful_exit` function so that we, well, exit gracefully.

Testing the Template

The finished template is a functional (and correct!) script. We can test it. First the help function:

```
me@linuxbox:~$ ./new_script-demo --help
new_script-demo ver. 0.1
Demonstrate the new_script template

Usage: new_script-demo [-h|--help]
       new_script-demo [-a|--option_a] [-b|--option_b b_argument]

  Options:
  -h, --help                  Display this help message and exit.
  -a, --option_a              The first option named 'a'
  -b, --option_b b_argument   The second option named 'b'
    Where 'b_argument' is the argument for option 'b'.

me@linuxbox:~$
```

With no options or arguments, the template produces no output.

```
me@linuxbox:~$ ./new_script-demo
me@linuxbox:~$
```

The template displays informative messages as it processes the options and arguments.

```
me@linuxbox:~$ ./new_script-domo -a
the first option named 'a'
me@linuxbox:~$ ./new_script-demo -b test
the second option named 'b'
b_argument == test
me@linuxbox:~$ ./new_script-demo ./*
Processing argument ./bin...
Processing argument ./Desktop...
Processing argument ./Disk_Images...
Processing argument ./Documents...
Processing argument ./Downloads...
    .
    .
    .
```

Summing Up

Using `new_script` saves a lot of time and effort. It's easy to use and it produces high quality script templates. Once a programmer decides on a script's options and arguments, they can use `new_script` to quickly produce a working script and add feature after feature until everything is fully implemented.

Feel free to examine the `new_script` code. Parts of it are exciting.

Further Reading

There are many bash shell script "templates" available on the Internet. A Google search for "bash script template" will locate some. Many are just small code snippets or suggestions on coding standards. Here are a few interesting ones worth reading:

- *Boilerpalte Shell Script Template* https://natelandau.com/boilerplate-shell-script-template/
- *Another Bash Script Template* https://jonlabelle.com/snippets/view/shell/another-bash-script-template
- *Basic script template for every bash script* https://coderwall.com/p/koixia/logging-mini-framework-snippet-for-every-shell-script
- *Argbash documentation* A template generator that works from a configuration file rather than interactively. https://argbash.readthedocs.io/en/latest/